AI-Based Crop & Agricultural Resource Optimization System

Voice-first AI advisor: tells a farmer WHICH crops to grow and HOW MANY ACRES of each to maximize profit within budget

VJH 2k26 Hackathon | Round 3: Final Jury Evaluation | Theme: AgriTech | Team: The_King | Team ID: AG_07
Leader: Pavan Kumar, IIIT Allahabad | iec2025049@iiita.ac.in

1 Project Overview

THE PROBLEM

- Agriculture supports **58%+ of India's rural households**, yet smallholders decide blind.
- Soil, weather and mandi-price data exist but are **scattered across separate portals**.
- Existing apps only answer "*what can grow here?*" and demand N-P-K / pH values farmers cannot provide.
- Nobody answers: **"How do I split 5 acres and Rs. 1,00,000 to earn the most without going broke?"**

OUR SOLUTION

A **full-stack, voice-enabled advisor** that needs only **3 simple inputs** and fuses **Machine Learning** (what is suitable) with **Operations Research** (how much to plant) and **Generative AI** (plain-language why). Output: a ready crop-wise acreage plan, budget split and net-profit balance sheet, spoken and shown in simple units (acres, quintals, Rs.).

3 FARMER INPUTS
Village / District
Land (Acres) | Budget (Rs.)

→

AUTO DATA FETCH
Weather 90-day | Soil
Mandi prices (22 crops)

→

AI + OPTIMIZATION
Random Forest + SHAP
Linear Programming (HiGHS)

→

FARMER GETS
Crop split (donut) + profit sheet
Gemini explanation + voice

3	22	99.3%	0.84	6
inputs only	crops covered	classifier accuracy	yield R^2 (out-of-time)	integrated modules

2 Project Objective and Goals

Objective: democratize ML + Operations Research for smallholders by turning raw public data into a **trustworthy, explainable, profit-maximizing farm plan** that a non-technical farmer can use by voice.

Goal	Target	How we achieve it
Accessibility	100% voice-operable	HTML5 Web Speech API, Indian units (acres, quintals), no jargon
Simplicity	Zero chemical inputs	Auto geocoding + SoilGrids / Soil Health Card; optional lab override
Accuracy	Top-3 suitable crops	Random Forest, 22 classes, 99.3% hold-out accuracy
Economic soundness	Budget never exceeded	Linear Programming: Σ(cost × acres) ≤ budget, positive net margin
Risk reduction	No monoculture	Cap: max 65% of land on any one crop
Explainability	Every decision auditable	TreeSHAP + LP constraint logging + Gemini plain-language reasons
Speed	Fast, cached pipeline	Async threads, TTL cache, speculative prefetch (1.9 s warm, 4.7 s cold)

3 Project Modules (6 integrated sub-systems)

1 Voice Frontend
- React 19 + Vite, Web Speech API
- Donut chart, profit bars, XAI cards
- Data-provenance badges

2 Geo & Climate Engine
- Village → lat/lon/district (Open-Meteo)
- 90-day temp, rain, humidity
- /api/prefetch warms caches

3 Soil & Price Resolver
- Override > SoilGrids > SHC > fallback
- Mandi prices: CEDA / Agmarknet
- Fallback tagged 'simulated'

4 ML + Yield Engine
- Random Forest (100 trees, 22 crops)
- TreeSHAP feature attribution
- District-yield lookup v3 (leak-free)

5 LP Optimizer
- SciPy linprog, HiGHS solver
- Maximize profit: land+budget+cap
- Hectare→acre, tonne→quintal

6 Gemini Explainer
- Gemini API, Gemma-4 fallback
- Evidence-grounded JSON reasons
- Explains crops in simple words

Data flow: Frontend (1) → FastAPI backend → Data modules (2, 3) → ML engine (4) → LP optimizer (5) → Gemini (6) → Dashboard
Every module is independently testable (pytest suites for API, data sources, cache, and full-pipeline integration).

Example farmer journey: from one spoken sentence to a profit plan

1 SPEAK
"*Pune, 5 acres, Rs. 1,00,000*"
typed or spoken in the local language

→

2 AUTO-FETCH
Weather (90 days), soil and mandi prices are resolved with **no lab report** needed

→

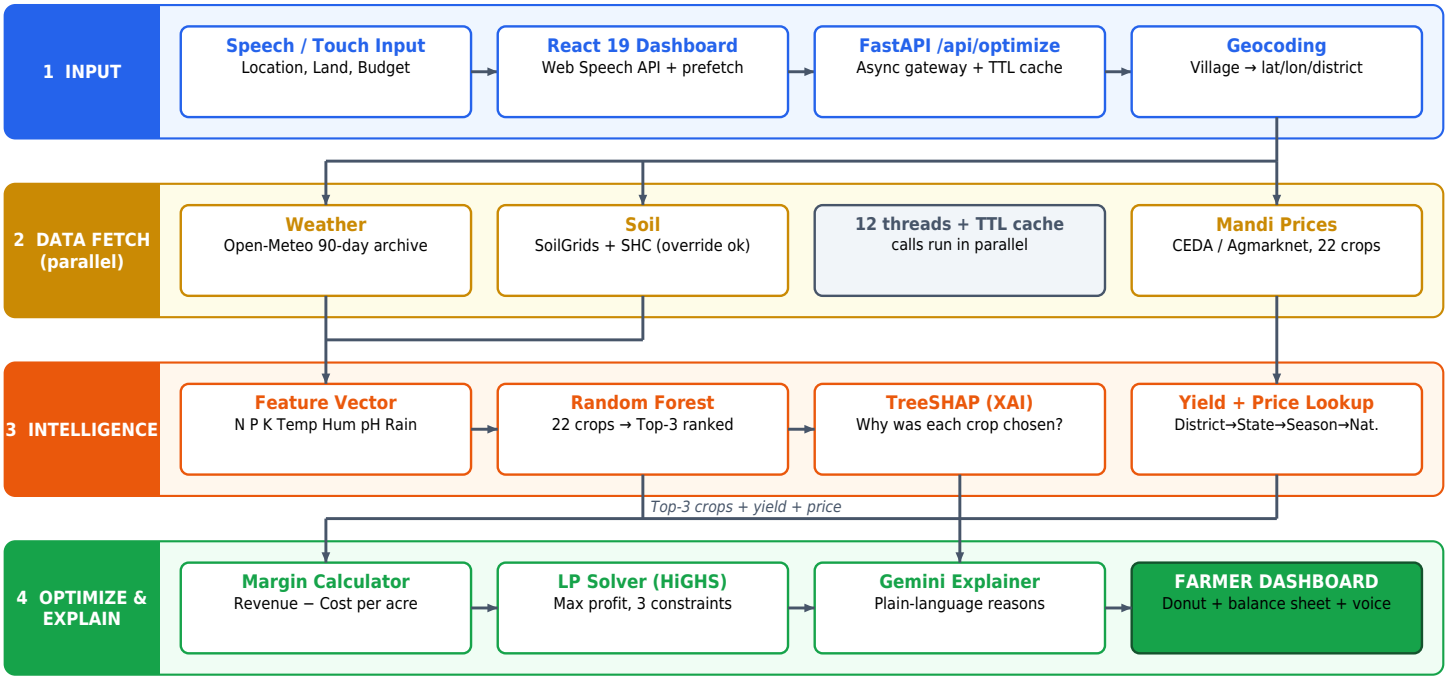
3 DECIDE
ML shortlists **top-3 crops**; Linear Programming splits the acres under budget and 65% cap

→

4 ACT
3.25 ac Jute + 0.92 ac Papaya
Net profit **Rs. 1,32,814**

Built for the farmer, auditable for the expert: every number on the dashboard is traceable to a data source (live / historical / simulated badge) and every recommendation carries a reason.

4 System Architecture and Workflow



Core Math: Linear Program (solved by SciPy HiGHS)

Let $\mathbf{x_i}$ = acres of crop *i* (top-3 crops). **L** = land (acres), **B** = budget (Rs.)

MAXIMIZE Profit = $\Sigma\ M_i \cdot x_i$

where margin M_i = ($Yield_i$ × $Price_i$) − $Cost_i$ per acre

1. **Land limit:** $\Sigma\ x_i \le L$
2. **Budget limit:** $\Sigma\ (Cost_i \cdot x_i) \le B$
3. **Risk cap:** $0 \le x_i \le 0.65 \cdot L$ (no monoculture)
4. **Units:** 1 ha = 2.471 acres, 1 tonne = 10 quintals

Result: provably optimal split in ~8 ms, with constraint slack logged for audit.

Yield Lookup v3: 4-level fallback

L1 District × Crop × Season	86.8% of queries
L2 State × Crop × Season	12.9%
L3 Crop × Season	0.3%
L4 National crop median	0.03%

5 Technology Stack and Implementation

Layer	Technology	Why we chose it
Frontend	React 19, Vite, Vanilla CSS, custom SVG charts	Fast UI, zero chart-library bloat, accessible and responsive
Voice	HTML5 Web Speech API	Hands-free input for low-literacy users, no paid API
Backend	FastAPI + Uvicorn, concurrent.futures (12 threads), custom TTLCache	Async, parallel API calls, stale-while-revalidate against rate limits
Machine Learning	Scikit-learn Random Forest, SHAP TreeExplainer	99.3% accuracy; exact, explainable feature attribution
Yield model	Hierarchical median lookup (crop_yield_lookup_v3.json)	Replaced XGBoost: leak-free, 61× faster, 95% smaller
Optimization	SciPy linprog (HiGHS simplex / interior-point)	Guaranteed optimum under land, budget, risk constraints
Generative AI	Google Gemini API (Gemma-4 fallback)	Structured, evidence-grounded explanations, no invented facts
Data sources	Open-Meteo, SoilGrids, Soil Health Card, CEDA / Agmarknet	Free, public, India-relevant; 246k govt records cleaned to 67k

Implementation highlights: 124 raw crop names mapped to 22 classes; data cleaned 1997-2020 (246,091 → 67,396 records); outlier pruning; 4 automated test suites; per-feature **provenance badges** (live / historical / simulated) so judges and farmers always know the data origin.

Soil data trust cascade: best available source wins, and the plan says which was used

1 Farmer lab test
Exact N, P, K, pH typed by the user.
Badge: verified

→

2 SoilGrids API
Mapped soil chemistry at the village coordinates.
Badge: live

→

3 Soil Health Card
District-level nutrient summary from the national portal.
Badge: historical

→

4 Local fallback
Reproducible estimate if public data fails.
Badge: simulated

Three layers of explainability: why the system chose what it chose

TreeSHAP (ML layer)
Shows which soil and climate features drove each crop. Pune demo: 880 mm rain added **+0.42** to Rice and **+0.31** to Jute.

LP constraint log (math layer)
Shows which limit was binding: Jute hit the **65% land cap**, Papaya used the **remaining budget**, Rice was dropped for negative margin.

Gemini (language layer)
Turns the numbers into one plain-language reason per crop, grounded only in the computed evidence.

6 Results and Demonstration

99.32%
Classifier accuracy
P 0.9935 | R 0.9932 | F1 0.9932

0.8361
Yield R² (out-of-time)
26,283 records, test years 2008-14

0.75 t/ha
Yield MAE
97.5% lower than XGBoost (29.33)

1.9 s / 4.7 s
End-to-end latency
warm cache / cold start

Live end-to-end test: Pune, Maharashtra | 5 acres | Rs. 1,00,000 | Kharif season | Soil auto-resolved (N80 P40 K45 pH6.5)

Crop (RF rank)	Yield q/acre	Price Rs./q	Cost Rs./acre	Net margin Rs./acre	LP acres
Jute (#2)	11.33	4,500	18,000	**+32,985**	**3.25** (cap)
Papaya (#3)	60.70	1,200	45,000	**+27,840**	**0.92**
Rice (#1)	8.90	2,183	20,000	**−572**	**0.00**

Key insight: Rice is the classifier's #1 pick but loses money at local prices, so the optimizer drops it. *Suitability alone is not profitability.*

Optimization Balance Sheet	Value
Land utilized	4.17 / 5.0 acres
Budget invested	Rs. 99,900 / 1,00,000
Gross revenue	Rs. 2,32,714
Cultivation cost	Rs. 99,900
NET PROFIT	**Rs. 1,32,814**

Land allocation of 5.0 acres



Why we replaced XGBoost: yield-model evolution (honest validation, no data leakage)

Model	Validation	MAE (t/ha)	R²	Latency	Size	Leakage
EXP_REG_01 XGBoost	Random 80/20	29.33	−45.2 (per-crop)	18.5 ms	42.8 MB	High
EXP_REG_02 State lookup	Random 80/20	0.908	0.811	0.5 ms	1.8 MB	None
EXP_REG_03 District lookup (live)	**Out-of-time 2008-14**	**0.747**	**0.836**	**0.3 ms**	**2.1 MB**	**Zero**

Finding: XGBoost looked good (R² 0.99) only because Yield = Production / Area leaked into its features; on causal pre-season features it collapsed. We caught this and rebuilt the model with strict time-ordered validation. **Per-crop lookup MAE:** Lentil 0.09, Blackgram 0.10, Chickpea 0.22, Rice 0.37, Maize 0.60 t/ha.

6.1 Latency and performance benchmarks

Pipeline stage	Cold (ms)	Warm (ms)	Speedup
Geocoding resolution	340.2	0.8	**425x**
90-day climate fetch	820.5	1.1	**745x**
Soil input resolution	210.4	0.5	**420x**
RF classifier + TreeSHAP	45.2	32.1	**1.4x**
District yield lookup v3	0.4	0.3	**1.3x**
Mandi price resolution	185.0	1.2	**154x**
SciPy HiGHS LP solver	8.5	6.2	**1.4x**
Gemini AI reasoning	3,120.0	1,890.0	**1.6x**
Total end-to-end	**4,730.2**	**1,932.2**	**2.4x**

6.2 Classifier performance (EXP_CLS_01)

Metric	Hold-out (n=440)	5-fold CV
Accuracy	**99.32%** (437/440)	99.26% ± 0.61%
Precision (macro)	**0.9935**	0.9926 ± 0.0062
Recall (macro)	**0.9932**	0.9926 ± 0.0062
F1-score (macro)	**0.9932**	0.9926 ± 0.0062
F1-score (weighted)	**0.9932**	0.9926 ± 0.0062

Top feature drivers (Gini): Rainfall 22.36%, Humidity 21.54%, Potassium 17.61%, Phosphorus 14.90%, Nitrogen 10.58%, Temperature 7.47%, pH 5.53%. 22 crop classes, 2,200 observations, 80/20 split.

Reading the tables: cached data APIs drop from 100s of ms to about 1 ms (up to 745x); the LP optimizer takes under 10 ms. The Gemini call dominates (3.1 s of 4.7 s cold). Precision, recall and F1 all stay at 0.99+ because the classes are well separated.

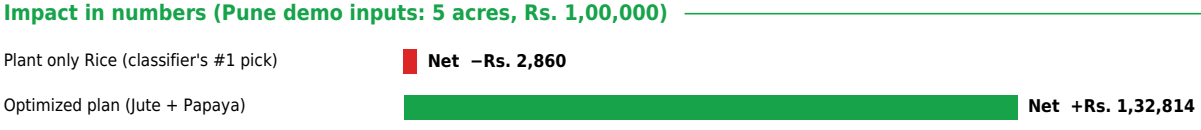
7 Innovation and Impact

WHAT IS NEW

1. Prediction → Prescription: ML says *what* grows; Linear Programming says *how many acres*, giving a real balance sheet.
2. Scientific integrity: leak-free, time-validated yield model; every data point badged live / historical / simulated.
3. Layered explainability: TreeSHAP + LP slack + Gemini in farmer-friendly words.
4. Built-in risk guardrail: 65% single-crop cap protects against pests and price crashes.
5. Voice-first, 3-input UX in acres, quintals and rupees.

Conventional approach	With our platform
Monoculture planting	Diversified 2-3 crop split
Bankruptcy / over-borrowing risk	Budget mathematically capped
Guesswork on mandi rate	Verified historical prices
Jargon-heavy portals, N-P-K inputs	Voice, 3 simple inputs
No profit planning	Optimal net-profit plan

Impact in numbers (Pune demo inputs: 5 acres, Rs. 1,00,000)



Illustrative: Rice-only uses the demo's own yield, price and cost (Rs. 97,140 revenue vs Rs. 1,00,000 cost).

8 Future Scope and Conclusion

PHASE 1: DONE
- Core ML + HiGHS LP solver
- Voice UI + Gemini explainer
- Working prototype

PHASE 2: NEXT
- IoT LoRaWAN soil sensors
- Sentinel-2 NDVI crop health
- Multi-year rotation (DP)

PHASE 3: ECOSYSTEM
- Direct mandi / FPO linkage
- Micro-credit with verified plans
- Parametric crop insurance

CONCLUSION
We built a working system that unifies **Random Forest + TreeSHAP**, **leak-free yield analytics**, **HiGHS Linear Programming** and **Gemini explanations** behind a voice-first interface. From just a location, land size and budget, a farmer receives a transparent, budget-safe, profit-maximizing crop plan. Our aim: move smallholders from guesswork to decision-grade, explainable AI.